CS 432 – Databases | Assignment 4: Sharding

Group: Track 1 — Sports Club Management System (IIT Gandhinagar)

GitHub Repository:
https://github.com/1357koushik/Sharkey_Database_Project/tree/main/Assignment_4

Video Demonstration:
https://drive.google.com/file/d/1GoRzNkXrI9J8PyBh5Om6ubmZJTbAC2M1/view?usp=sharing

---

---

1. Project Overview

This assignment extends the **Sports Club Managment** database with **horizontal scaling via sharding**.

Member-centric tables include:

Table	Rows	Shard Key
Member	40	Member_ID
Player	20	Member_ID
Coach	20	Member_ID
Administrator	10	Member_ID
Booking	20	Member_ID
Attendance	20	Member_ID
Equipment_Loan	20	Member_ID
Complaint	20	Raised_By (Member_ID)
Player_Stat	20	Member_ID
Team_Roster	20	Member_ID (via Player)

Reference tables (Sport, Facility, Equipment, Team, Event) are **replicated** to every shard.

---

2. Shard Key Selection & Justification

Shard Key: `Member_ID`

Criterion	Assessment
High Cardinality	40 distinct IDs now grows with every new member registration. The numeric suffix (1–∞) ensures unbounded distinct values.
Query-Aligned	Every API endpoint that reads or mutates member data uses `Member_ID` in its routing logic for members, bookings, attendance, complaints, and stats.
Stable	`Member_ID` is assigned at registration and never changes. It is the primary key of the `Member` table.

Partitioning Strategy: Hash-Based

```
shard_id = int(member_id[1:]) % 3
```

Why hash-based over range or directory?
- **Hash-based** is O(1), deterministic, requires no extra storage, and distributes new IDs uniformly across all shards.

Expected Distribution

Shard	Member_IDs	Count
Shard 1 (index 0)	M03, M06, M09, M12, M15, M18, M21, M24, M27, M30, M33, M36, M39	13
Shard 2 (index 1)	M01, M04, M07, M10, M13, M16, M19, M22, M25, M28, M31, M34, M37, M40	14
Shard 3 (index 2)	M02, M05, M08, M11, M14, M17, M20, M23, M26, M29, M32, M35, M38	13

Maximum skew = 14 − 13 = **1 row**.

Shard Isolation Simulation

Each shard is an independent **Assignment 3 engine instance** (B+Tree + WAL), and routing is enforced by the query router.
Handout network details are documentation-only and are not used for runtime storage.

In [3]:
```
import os
import sys

root = os.path.abspath(".")
router_path = os.path.join(root, "router")
if not os.path.isdir(router_path):
    router_path = os.path.join(root, "Assignment_4", "router")
sys.path.insert(0, router_path)

from shard_config import expected_distribution, NUM_SHARDS, SHARD_PATHS

all_ids = [f"M{i:02d}" for i in range(1, 41)]
dist = expected_distribution(all_ids)

print(f"Hash function : shard_id = int(member_id[1:]) % {NUM_SHARDS}")
print("Shard paths   :")
for sid, path in SHARD_PATHS.items():
    print(f"  shard_{sid} -> {path}")
print()
for sid, ids in dist.items():
    print(f"  Shard {sid + 1} ({len(ids):2d} members): {ids}")
```

```
Hash function : shard_id = int(member_id[1:]) % 3
Shard paths   :
  shard_0 -> assignment3-engine://shard_db_0
  shard_1 -> assignment3-engine://shard_db_1
  shard_2 -> assignment3-engine://shard_db_2

  Shard 1 (13 members): ['M03', 'M06', 'M09', 'M12', 'M15', 'M18',
'M21', 'M24', 'M27', 'M30', 'M33', 'M36', 'M39']
  Shard 2 (14 members): ['M01', 'M04', 'M07', 'M10', 'M13', 'M16',
'M19', 'M22', 'M25', 'M28', 'M31', 'M34', 'M37', 'M40']
  Shard 3 (13 members): ['M02', 'M05', 'M08', 'M11', 'M14', 'M17',
'M20', 'M23', 'M26', 'M29', 'M32', 'M35', 'M38']
```

SubTask 2: Shard Setup and Data Migration

Schema Initialisation

Each shard database receives:
- **Reference tables** (exact copies, replicated): Sport, Facility, Equipment, Team, Event
- **Partitioned tables** (prefixed `shard_N_`): member, player, coach, administrator, booking, attendance, equipment_loan, complaint, player_stat, team_roster

The prefix naming convention follows the assignment specification: `Member`, `Booking`, `Complaint`, etc.

In [4]:
```
from router.init_shards import init_all_shards
from shard_config import NUM_SHARDS, SHARD_DATABASES, get_shard_table

init_all_shards()

for sid in range(NUM_SHARDS):
    print(f"\nShard {sid + 1} store: {SHARD_DATABASES[sid]}")
    print("  Tables: Member, Facility, Booking, Complaint, Attendance,
Player_Stat")
    print(f"  Member rows: {len(list(get_shard_table(sid,
'Member').get_all()))}")
```

```
  [OK] Shard 1 initialised on assignment3-engine-shard-1
  [OK] Shard 2 initialised on assignment3-engine-shard-2
  [OK] Shard 3 initialised on assignment3-engine-shard-3
All 3 Assignment 3 engine shards ready.

Shard 1 store: shard_db_0
  Tables: Member, Facility, Booking, Complaint, Attendance,
Player_Stat
  Member rows: 1

Shard 2 store: shard_db_1
  Tables: Member, Facility, Booking, Complaint, Attendance,
Player_Stat
  Member rows: 3

Shard 3 store: shard_db_2
  Tables: Member, Facility, Booking, Complaint, Attendance,
Player_Stat
  Member rows: 3
```

Data Migration

In [5]:
```
from router.migrate_data import run_migration
run_migration()
```

```
-- Partitioning summary ------------------------------------
  [PART] Member: 7 rows  ->  shard_0:1  shard_1:3  shard_2:3
  [PART] Booking: 3 rows  ->  shard_0:1  shard_1:1  shard_2:1
  [PART] Complaint: 3 rows  ->  shard_0:1  shard_1:1  shard_2:1
  [PART] Attendance: 2 rows  ->  shard_0:0  shard_1:1  shard_2:1
  [PART] Player_Stat: 2 rows  ->  shard_0:0  shard_1:1  shard_2:1
  [REF] Facility: 5 rows replicated to all 3 shards
Migration complete.
```

Verification: Data Integrity

No Records Lost — Row Count Comparison

For every partitioned table: `source_count == sum(shard_1_count + shard_2_count + shard_3_count)`

In [6]:
```
from shard_config import get_shard_table, NUM_SHARDS

tables_expected = {
    "Member": 7,
    "Booking": 3,
    "Complaint": 3,
}

print(f"{'Table':<20} {'Expected':>8} {'S0':>6} {'S1':>6} {'S2':>6}
{'Total':>8} {'OK':>5}")
print("-" * 70)
all_ok = True
for table_name, expected in tables_expected.items():
    shard_counts = []
    for sid in range(NUM_SHARDS):
        n = len(list(get_shard_table(sid, table_name).get_all()))
        shard_counts.append(n)
    total = sum(shard_counts)
    ok = total == expected
    all_ok = all_ok and ok
    print(f"{table_name:<20} {expected:>8} {shard_counts[0]:>6}
{shard_counts[1]:>6} {shard_counts[2]:>6} {total:>8} {str(ok):>5}")

print("Overall:", "PASS ✅" if all_ok else "FAIL ❌")
```

```
Table                Expected     S0     S1     S2    Total    OK
----------------------------------------------------------------------
----
Member                      7      1      3      3        7  True
Booking                     3      1      1      1        3  True
Complaint                   3      1      1      1        3  True
Overall: PASS ✅
```

4.2 No Duplicates Across Shards

In [7]:
```
from shard_config import NUM_SHARDS, get_shard_table

all_member_ids = []
for sid in range(NUM_SHARDS):
    rows = [row["Member_ID"] for _, row in get_shard_table(sid,
"Member").get_all()]
    all_member_ids.extend(rows)

total = len(all_member_ids)
unique = len(set(all_member_ids))
dupes = sorted({x for x in all_member_ids if all_member_ids.count(x) >
1})
print(f"Total Member rows across shards : {total}")
print(f"Unique Member_IDs               : {unique}")
print(f"Duplicate IDs                   : {dupes if dupes else 'None'}")
```

```
Total Member rows across shards : 7
Unique Member_IDs               : 7
Duplicate IDs                   : None
```

4.3 Every Record Is in Its Correct Shard

In [8]:
```
from shard_config import get_shard_id, NUM_SHARDS, get_shard_table

misplaced = []
for sid in range(NUM_SHARDS):
    rows = [row for _, row in get_shard_table(sid, "Member").get_all()]
    for row in rows:
        mid = row["Member_ID"]
        expected = get_shard_id(mid)
        if expected != sid:
            misplaced.append((mid, sid, expected))

print("Checked all Member rows for correct shard placement.")
print(f"Misplaced records: {len(misplaced)}")
if not misplaced:
    print("Every record is in its hash-correct shard.")
else:
    for mid, found, expected in misplaced:
        print(f"MISPLACED: {mid} found in shard {found}, expected shard
{expected}")

print("\nSpot checks (Member_ID -> expected shard):")
for mid in ["M01", "M02", "M03", "M40", "M30"]:
    actual = get_shard_id(mid)
    print(f"  {mid}: int('{mid[1:]}') % 3 = {actual}")
```

```
Checked all Member rows for correct shard placement.
Misplaced records: 0
Every record is in its hash-correct shard.

Spot checks (Member_ID -> expected shard):
  M01: int('01') % 3 = 1
  M02: int('02') % 3 = 2
  M03: int('03') % 3 = 0
  M40: int('40') % 3 = 1
  M30: int('30') % 3 = 0
```

SubTask 3: Query Routing

The `query_router.py` module implements the three required routing patterns.
All routing is O(1) — no lookup table, no inter-shard communication for lookups.

Lookup Queries — Single Shard

In [9]:
```
from router.query_router import get_member, get_member_bookings,
get_member_complaints

print("=== Lookup: get_member('M01') ===")
m = get_member('M01')
print(f"  Name      : {m['Name']}")
print(f"  Age       : {m['Age']}")
print(f"  Routed to : shard_{m['_routed_to_shard']}")
print(f"  Formula   : int('01') % 3 = {1} ")

print()
print("=== Lookup: get_member('M03') ===")
m = get_member('M03')
print(f"  Name      : {m['Name']}")
print(f"  Routed to : shard_{m['_routed_to_shard']}")
print(f"  Formula   : int('03') % 3 = {0} ")

print()
print("=== Lookup: get_member('M02') ===")
m = get_member('M02')
print(f"  Name      : {m['Name']}")
print(f"  Routed to : shard_{m['_routed_to_shard']}")
print(f"  Formula   : int('02') % 3 = {2} ")

print()
print("=== Lookup: get_member_bookings('M01') ===")
bookings = get_member_bookings('M01')
shards_used = set(b['_shard'] for b in bookings)
print(f"  Bookings  : {len(bookings)}")
print(f"  Shards used: {shards_used}  (only shard_1 — single-shard
lookup )")
print()
for b in bookings:
    print(f"  Booking_ID={b['Booking_ID']}  Facility={b['Facility_ID']}
Time_In={b['Time_In']}")
```

```
=== Lookup: get_member('M01') ===
  Name      : Rahul Sharma
  Age       : 18
  Routed to : shard_2
  Formula   : int('01') % 3 = 1

=== Lookup: get_member('M03') ===
  Name      : Aman Verma
  Routed to : shard_1
  Formula   : int('03') % 3 = 0

=== Lookup: get_member('M02') ===
  Name      : Neha Singh
  Routed to : shard_3
  Formula   : int('02') % 3 = 2
```

```
=== Lookup: get_member_bookings('M01') ===
  Bookings  : 1
  Shards used: {2}  (only shard_1 — single-shard lookup )

  Booking_ID=1  Facility=1  Time_In=2026-02-01 13:30:00
```

Insert Operations — Routed to Correct Shard

In [10]:

```
from router.query_router import insert_member, insert_booking
from shard_config import NUM_SHARDS, get_shard_table

res = insert_member({
    "Member_ID": "M41",
    "Name": "Test Member FortyOne",
    "Gender": "M",
    "Email": "test_m41_demo@iitgn.ac.in",
    "Age": 23,
})
print(f"insert_member('M41'): {res}")

for sid in range(NUM_SHARDS):
    row = get_shard_table(sid, "Member").get("M41")
    symbol = "FOUND" if row else "absent"
    print(f"  shard_{sid}: {symbol}")

b_res = insert_booking({"Member_ID": "M41", "Facility_ID": 1, "Time_In":
"2026-04-18 10:00:00"})
print(f"\ninsert_booking for M41: {b_res}")

member_sid = 41 % 3
booking_table = get_shard_table(member_sid, "Booking")
for key, row in list(booking_table.get_all()):
    if row.get("Member_ID") == "M41":
        booking_table.delete(key)

member_table = get_shard_table(member_sid, "Member")
if member_table.get("M41"):
    member_table.delete("M41")

print("(Test records cleaned up)")
```

```
insert_member('M41'): {'ok': True, 'shard_index': 2, 'shard': 3,
'Member_ID': 'M41'}
  shard_0: absent
  shard_1: absent
  shard_2: FOUND

insert_booking for M41: {'ok': True, 'shard_index': 2, 'shard': 3,
'Booking_ID': 4}
(Test records cleaned up)
```

Range Queries — Scatter to All Shards, Merge Results

In [11]:
```
from router.query_router import range_members_by_age,
range_bookings_by_date, range_attendance_by_date

print(" Range Query: Members aged 18–21 ")
print()
print("Query touches ALL 3 shards (members are distributed by hash)")
members = range_members_by_age(18, 21)
shards_touched = set(m['_shard'] for m in members)
print(f"Shards queried : {sorted(shards_touched)}")
print(f"Members found  : {len(members)}")
for m in members:
    print(f"  {m['Member_ID']}  {m['Name']:<20}  age={m['Age']}
shard_{m['_shard']}")
```

```
 Range Query: Members aged 18–21

Query touches ALL 3 shards (members are distributed by hash)
Shards queried : [1, 2, 3]
Members found  : 4
  M01  Rahul Sharma          age=18  shard_2
  M02  Neha Singh            age=19  shard_3
  M03  Aman Verma            age=20  shard_1
  M04  Priya Mehta           age=21  shard_2
```

In [12]:
```
print("Range Query: Bookings 2026-02-01 to 2026-02-10")
print()
bookings = range_bookings_by_date("2026-02-01", "2026-02-10")
shards_touched = set(b['_shard'] for b in bookings)
print(f"Shards queried : {sorted(shards_touched)}")
print(f"Bookings found : {len(bookings)}  (merged and sorted by
Time_In)")
for b in bookings:
    print(f"  Booking_ID={b['Booking_ID']:>2}  Member={b['Member_ID']}
"
          f"Time_In={b['Time_In']}  shard_{b['_shard']}")
```

```
Range Query: Bookings 2026-02-01 to 2026-02-10

Shards queried : [1, 2, 3]
Bookings found : 3  (merged and sorted by Time_In)
  Booking_ID= 1  Member=M01  Time_In=2026-02-01 13:30:00  shard_2
  Booking_ID= 2  Member=M02  Time_In=2026-02-02 14:30:00  shard_3
  Booking_ID= 3  Member=M03  Time_In=2026-02-03 15:30:00  shard_1
```

---

6. Full Automated Verification

In [13]:

```
import subprocess
import sys
import os
from pathlib import Path


def run_script(folder_path, script_name):
    folder = Path(folder_path).resolve()
    script = folder / script_name

    if not script.exists():
        print(f"ERROR: File not found -> {script}")
        return

    env = {**os.environ, "PYTHONIOENCODING": "utf-8"}
    result = subprocess.run(
        [sys.executable, str(script)],
        cwd=folder,
        capture_output=True,
        text=True,
        encoding="utf-8",
        errors="replace",
        env=env,
    )

    print("=== STDOUT ===")
    print(result.stdout if result.stdout else "[empty]")
    print("=== STDERR ===")
    print(result.stderr if result.stderr else "[empty]")
    print(f"=== EXIT CODE: {result.returncode} ===")


base = Path.cwd()
if (base / "tests" / "verify_sharding.py").exists():
    target = base / "tests"
elif (base / "Assignment_4" / "tests" / "verify_sharding.py").exists():
    target = base / "Assignment_4" / "tests"
else:
    target = base / "tests"

run_script(target, "verify_sharding.py")
```

```
=== STDOUT ===
[RECOVERY] WAL is empty — nothing to recover.
Database 'shard_db_0' created.
Table 'Member' created in database 'shard_db_0'.
Table 'Facility' created in database 'shard_db_0'.
Table 'Booking' created in database 'shard_db_0'.
Table 'Complaint' created in database 'shard_db_0'.
Table 'Attendance' created in database 'shard_db_0'.
Table 'Player_Stat' created in database 'shard_db_0'.
[RECOVERY] WAL is empty — nothing to recover.
Database 'shard_db_1' created.
Table 'Member' created in database 'shard_db_1'.
Table 'Facility' created in database 'shard_db_1'.
Table 'Booking' created in database 'shard_db_1'.
```

```
Table 'Complaint' created in database 'shard_db_1'.
Table 'Attendance' created in database 'shard_db_1'.
Table 'Player_Stat' created in database 'shard_db_1'.
[RECOVERY] WAL is empty — nothing to recover.
Database 'shard_db_2' created.
Table 'Member' created in database 'shard_db_2'.
Table 'Facility' created in database 'shard_db_2'.
Table 'Booking' created in database 'shard_db_2'.
Table 'Complaint' created in database 'shard_db_2'.
Table 'Attendance' created in database 'shard_db_2'.
Table 'Player_Stat' created in database 'shard_db_2'.

== A. Routing Formula ========================================
  [92mPASS[0m  M01 -> remainder 1 (handout Shard 2)
  [92mPASS[0m  M03 -> remainder 0 (handout Shard 1)
  [92mPASS[0m  M38 -> remainder 2 (handout Shard 3)
  [92mPASS[0m  Distribution skew <= 1
         sizes=[13, 14, 13]

== B. Assignment3 Engine Shards ==============================
  [92mPASS[0m  shard_0 member table available
         rows=1
  [92mPASS[0m  shard_1 member table available
         rows=3
  [92mPASS[0m  shard_2 member table available
         rows=3

== C. Lookup Routing =========================================
  [92mPASS[0m  get_member('M01') returns row from routed shard
         {'Member_ID': 'M01', 'Name': 'Rahul Sharma', 'Gender':
'M', 'Email': 'user1@iitgn.ac.in', 'Phone_Number': '900000001',
'Age': 18, 'DOB': '', 'Image': '', '_routed_to_shard_index': 1,
'_routed_to_shard': 2}

== SUMMARY ====================================================
  8/8 checks passed   0 failed

=== STDERR ===
[empty]
=== EXIT CODE: 0 ===
```

7. SubTask 4: Scalability & Trade-offs Analysis

Horizontal vs. Vertical Scaling

Vertical scaling means upgrading a single database server (more CPU/RAM/storage). It is simple, but eventually capped by hardware and remains a single failure domain.

Horizontal scaling (sharding) distributes data across multiple nodes. Capacity increases by adding nodes rather than only upgrading one node. In this system, data is split across three shards, each handling a portion of member-related workload based on the hash routing formula.

As the club grows, distribution remains deterministic via:

```
int(member_id[1:]) % NUM_SHARDS
```

Consistency

Single-shard operations: ACID behavior is supported by the Assignment 3 engine through WAL-backed transactions.

Cross-shard operations: No distributed transaction protocol (like 2PC) is implemented; multi-shard workflows require explicit orchestration to avoid partial-failure inconsistency.

Replicated reference tables: Reference data (like `Facility`) is copied to all shards. Updates must be propagated to each shard, creating temporary eventual-consistency windows if updates are not atomic across shards.

Availability

Single-shard failure: Data mapped to that shard becomes unavailable while other shards can still serve routed requests.

Lookup queries: continue for unaffected shards.

Range queries: can be partial if one shard is unavailable unless retry/fallback logic is

added.

Partition Tolerance

Routing is stateless and deterministic, so recovery after shard restoration is straightforward.

Inserts targeting a failed shard should fail explicitly or be queued for retry.

Observations and Limitations

- Re-sharding cost: adding shards requires data redistribution.
- Cross-shard joins: need scatter-gather and in-memory merge.
- Runtime storage in this implementation uses Assignment 3 engine shards.
- Handout network details are documentation-only in this runtime implementation.

---

8. Project File Structure

```
assignment4/
├── router/
│  ├── shard_config.py # Engine shard topology and routing helpers
│  ├── init_shards.py # Initializes Assignment 3 engine shard state
│  ├── migrate_data.py # Repartitions demo data across three shards
│  └── query_router.py # Lookup / insert / range routing functions
├── logs/
│  └── shard_*.wal.log # WAL files for shard engine instances
├── tests/
│  └── verify_sharding.py # Automated routing verification
├── sharded_app.py # Shard-aware Flask API
└── CS432_Assignment4_Report.ipynb
```

Exported with runcell — convert notebooks to HTML or PDF anytime at runcell.dev.